

SMD - Software Modeling and Design Unit 1 Notes

Micro and Macro Development Process of Grady Booch

Macro Development Process:

1. Conceptualization – Understand problem and requirements.
2. Analysis – Identify objects, classes and relationships.
3. Design – Create architecture and class structure.
4. Evolution – Coding and implementation.
5. Maintenance – Error fixing and enhancement.

Micro Development Process:

1. Identify classes and objects.
 2. Define semantics (attributes and methods).
 3. Identify relationships.
 4. Implement classes.
 5. Test and refine repeatedly.
-

Rumbaugh's Object Modeling Technique (OMT)

OMT was developed by James Rumbaugh. It uses three models:

1. **Object Model:** Represents static structure using classes and objects.
2. **Dynamic Model:** Represents system behavior using states and events.
3. **Functional Model:** Represents data processing using DFDs.

Phases: Analysis, System Design, Object Design, Implementation.

Object-Oriented Analysis by Coad–Yourdon

Coad–Yourdon method identifies objects and relationships from requirements.

Main layers:

1. Classes & Objects
2. Structures (Relationships)
3. Subjects (Grouping classes)
4. Attributes
5. Services (Methods)

Advantages: Modular design, easy understanding, reduced complexity.

Object-Oriented Software Engineering (OOSE) by Ivar Jacobson

OOSE focuses on use cases and user interaction.

Main phases:

1. Requirement Model
2. Analysis Model
3. Design Model
4. Implementation Model
5. Test Model

Important concepts: Use Cases, Actors, Entity Objects, Boundary Objects, Control Objects.

Unified Approach in Modeling

Unified Approach combines Booch, Rumbaugh and Jacobson methodologies.

Booch contributes design methods.

Rumbaugh contributes modeling techniques.

Jacobson contributes use case modeling.

Process:

Requirements → OOA → OOD → Implementation → Testing.

Significance of Design Methodologies

1. Provides structured development.
 2. Improves software quality.
 3. Reduces complexity.
 4. Enhances communication.
 5. Supports reusability.
 6. Easier maintenance.
 7. Reduces development cost and time.
-

Layered Approach to Software Development

Software is divided into layers:

1. Presentation Layer
2. Application Layer
3. Business Logic Layer
4. Data Access Layer
5. Database Layer

Advantages: Modularity, maintainability, scalability.

Structural Things in UML

Structural things represent static parts of a system.

Types:

1. Class

2. Object
 3. Interface
 4. Collaboration
 5. Use Case
 6. Active Class
 7. Component
 8. Node
-

Building Blocks in UML

UML building blocks are:

1. Things (Structural, Behavioral, Grouping, Annotational)
 2. Relationships (Association, Dependency, Generalization, Realization)
 3. Diagrams (Class, Use Case, Sequence, Activity, etc.)
-

Four Common Mechanisms of UML

1. Specifications – Detailed information about elements.
 2. Adornments – Extra symbols and notations.
 3. Common Divisions – Classifier vs Instance, Interface vs Implementation.
 4. Extensibility Mechanisms – Stereotypes, Tagged Values, Constraints.
-

Multiplicity in Association Relationship

Multiplicity defines number of objects participating in relationships.

Types:

1. One-to-One (1:1)
 2. One-to-Many (1:*)
 3. Many-to-One (*:1)
 4. Many-to-Many (*:*)
-

Class, Association Name, Association End Name and Multiplicity

Class: Blueprint of objects.

Association Name: Describes relationship between classes.

Association End Name: Specifies role played by classes.

Multiplicity: Specifies number of object instances in relationship.

UML Diagram Notational Techniques

Important UML notations:

1. Class notation
 2. Object notation
 3. Interface notation
 4. Association notation
 5. Aggregation notation
 6. Composition notation
 7. Generalization notation
 8. Dependency notation
 9. Use Case notation
-

4+1 View Architecture

Developed by Philippe Kruchten.

Views:

1. Logical View
 2. Process View
 3. Development View
 4. Physical View
- +1 Scenario (Use Case) View

Used for describing software architecture from multiple perspectives.

Iterative Development & Continuous Testing

Iterative Development: Software is developed in repeated cycles with continuous improvement.

Continuous Testing: Testing is performed during every stage of development to detect defects early.

Benefits: Better quality, reduced risk, early feedback.